

System Verilog based Function Verification of 11011 sequence detector (overlapping/non-overlapping)

Team Members:

Sangeerthana (2023BEC0044)

Amruth (2023BEC0050)

Faculty Mentor: Dr. Lakshmi NS

March 24, 2026

Contents

1	Overview of Sequence Detector	2
2	Project Workflow	5
3	RTL Design for DUT	8
4	Verification Concepts and Their Implementation	14

1 Overview of Sequence Detector

Our reference is <https://yue-guo.com/2020/01/14/sequence-detector-11011-moore-machine-mealy-machine-overlapping-non-overlapping/>

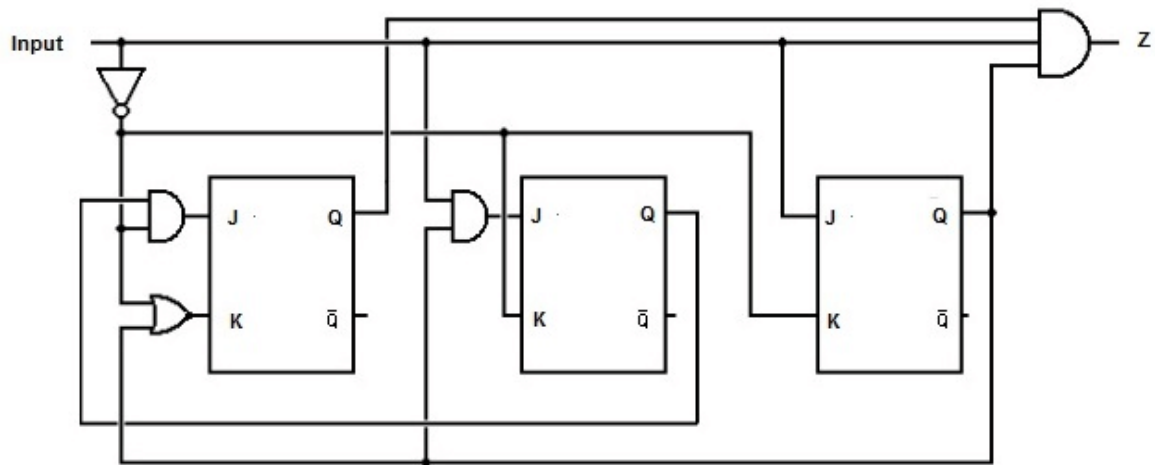
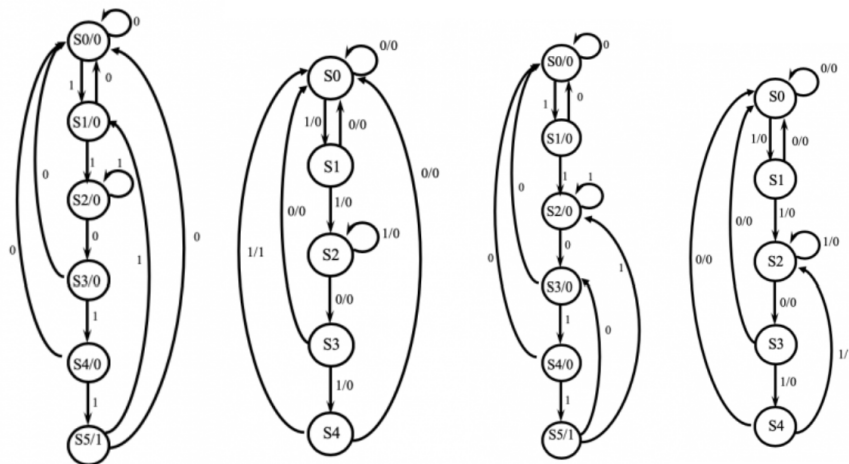


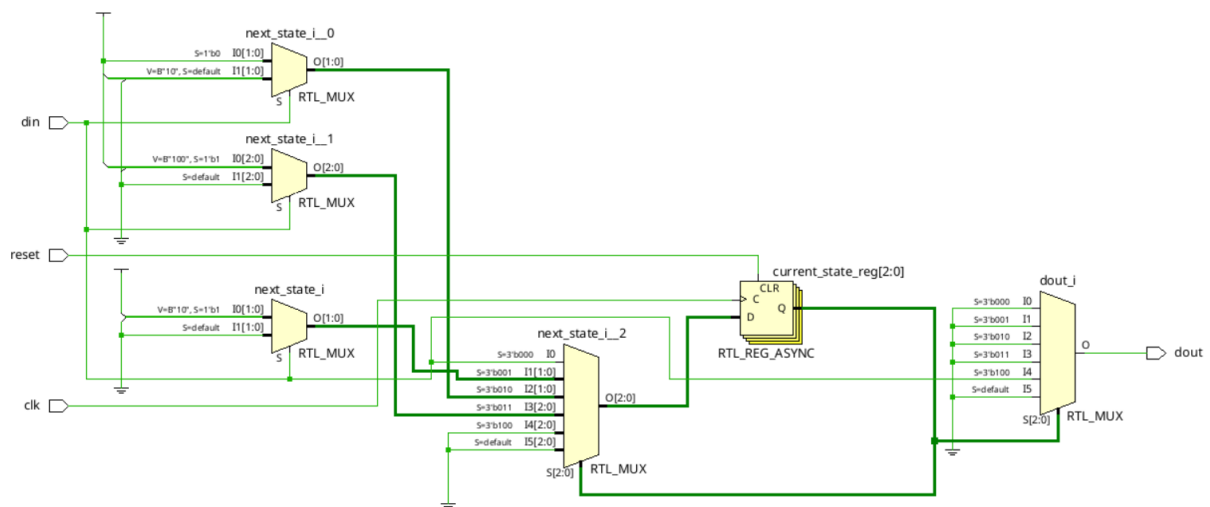
Figure 1: 11011 Sequence Detector Circuit

Below are FSM design for

Moore Machine (Non-Overlapping) , Mealy Machine (Non-Overlapping), Moore Machine (Overlapping) and Mealy Machine (Overlapping)



Schematic from Vivado :



The RTL :

```

module sequence_detector_11011_mealy_non_overlap (
    input clk,
    input reset,
    input din,
    output reg dout
);

    localparam [2:0]
        S0 = 3'b000,
        S1 = 3'b001,
        S2 = 3'b010,
        S3 = 3'b011,
        S4 = 3'b100;

    reg [2:0] current_state, next_state;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            current_state <= S0;
        end else begin
            current_state <= next_state;
        end
    end

    always @(*) begin
        next_state = current_state;
        dout = 1'b0;
    end

```

```
case (current_state)
  S0: begin
    if (din) next_state = S1;
    else     next_state = S0;
  end

  S1: begin
    if (din) next_state = S2;
    else     next_state = S0;
  end

  S2: begin
    if (~din) next_state = S3;
    else      next_state = S2;
  end

  S3: begin
    if (din) next_state = S4;
    else     next_state = S0;
  end

  S4: begin
    if (din) begin
      dout = 1'b1;

      next_state = S0;
    end else begin
      next_state = S0;
    end
  end

  default: next_state = S0;
endcase
end

endmodule
```

2 Project Workflow

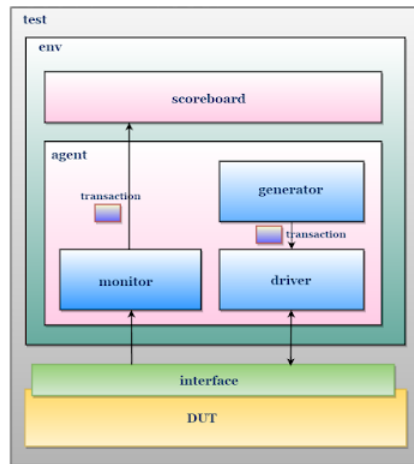


Figure 2: System Verilog Testbench Hierarchy

Project File Structure:

```

__testbench_top.sv
|
__sequence_detector_mealy_non_overlapping.sv
|
|__run.do
|
|__test.sv
|
|__env.sv
|
|__transaction.sv
|
|__scoreboard.sv
|
|__agent.sv
|
|__generator.sv
|
|__driver.sv
|
|__monitor.sv
  
```

The file structure :

```
|__run.do (Execution Script)
|
|__fsm_pkg.sv (Global State Definitions)
|
|__testbench.sv (Top-Level Module)
|
|__design.sv (DUT)
|
|__interface.sv
|
|__env.sv
|
|__transaction.sv
|
|__generator.sv
|
|__driver.sv
|
|__monitor.sv
|
|__scoreboard.sv
```

Important Note:

- The standard industry practice is to put the **covergroup** inside the **Monitor** (`monitor.sv`).
- The standard industry practice is to put the **assertions** inside the **Interface** (`interface.sv`).
- The standard industry practice is to put the **fork ... join** inside the **Environment** (`env.sv`).
- The standard industry practice is to put the **constraints** inside the **Transaction** (`transaction.sv`).

Order of writing code :

1. fsm-pkg.sv
2. design.sv
3. interface.sv
4. transaction.sv
5. generator.sv
6. driver.sv
7. monitor.sv
8. scoreboard.sv
9. env.sv
10. testbench.sv
11. run.do

3 RTL Design for DUT

1) Non-Overlapping Moore Sequence Detector of 11011

```
module sd11011_moore(input bit clk,
                    input logic reset,
                    input logic din,
                    output logic dout);

typedef enum logic [2:0] {S0, S1, S2, S3, S4, S5} state_t;
state_t state;

always @(posedge clk or posedge reset) begin
    if(reset) begin
        dout <= 1'b0;
        state <= S0;
    end
    else begin
        case(state)
            S0: begin
                dout <= 1'b0;
                if(din)
                    state <= S1;
            end
            S1: begin
                dout <= 1'b0;
                if(din)
                    state <= S2;
                else
                    state <= S0;
            end
            S2: begin
                dout <= 1'b0;
                if(~din)
                    state <= S3;
            end
            S3: begin
                dout <= 1'b0;
                if(din)
                    state <= S4;
                else
                    state <= S0;
            end
            S4: begin
                dout <= 1'b0;
                if(din)
                    state <= S5;
                else
                    state <= S0;
            end
        endcase
    end
end
```

```

        state <= S0;
    end
S5: begin
    dout <= 1'b1;
    if(din)
        state <= S1;
    else
        state <= S0;
    end
endcase
end
end

```

```
endmodule
```

2) Non-Overlapping Mealy Sequence Detector of 11011

```

module sd11011_mealy(input bit clk,
                    input logic reset,
                    input logic din,
                    output logic dout);

    typedef enum logic [2:0] {S0, S1, S2, S3, S4} state_t;
    state_t state;

    always @(posedge clk or posedge reset) begin
        if(reset) begin
            dout <= 1'b0;
            state <= S0;
        end
        else begin
            case(state)
                S0: begin
                    if(din) begin
                        state <= S1;
                        dout <=1'b0;
                    end
                    else
                        dout <=1'b0;
                end
                S1: begin
                    if(din) begin
                        state <= S2;
                        dout <=1'b0;
                    end

```

```

        else begin
            state <= S0;
            dout <=1'b0;
        end
    end
end
S2: begin
    if(~din) begin
        state <= S3;
        dout <=1'b0;
    end
    else begin
        dout <=1'b0;
    end
end
S3: begin
    if(din) begin
        state <= S4;
        dout <=1'b0;
    end
    else begin
        state <= S0;
        dout <=1'b0;
    end
end
S4: begin
    if(din) begin
        state <= S0;
        dout <=1'b1;
    end
    else begin
        state <= S0;
        dout <=1'b0;
    end
end
endcase
end
end
endmodule

```

3) Overlapping Moore Sequence Detector of 11011

```

module sd11011_moore_over(input bit clk,
                          input logic reset,
                          input logic din,
                          output logic dout);

```

```
typedef enum logic [2:0] {S0, S1, S2, S3, S4, S5} state_t;
state_t state;

always @(posedge clk or posedge reset) begin
    if(reset) begin
        dout <= 1'b0;
        state <= S0;
    end
    else begin
        case(state)
            S0: begin
                dout <= 1'b0;
                if(din)
                    state <= S1;
            end
            S1: begin
                dout <= 1'b0;
                if(din)
                    state <= S2;
                else
                    state <= S0;
            end
            S2: begin
                dout <= 1'b0;
                if(~din)
                    state <= S3;
            end
            S3: begin
                dout <= 1'b0;
                if(din)
                    state <= S4;
                else
                    state <= S0;
            end
            S4: begin
                dout <= 1'b0;
                if(din)
                    state <= S5;
                else
                    state <= S0;
            end
            S5: begin
                dout <= 1'b1;
                if(din)
                    state <= S2;
                else
                    state <= S3;
            end
        endcase
    end
end
```

```

        end
    endcase
end
end

```

```

endmodule

```

4) Overlapping Mealy Sequence Detector of 11011

```

module sd11011_mealy_over(input bit clk,
                        input logic reset,
                        input logic din,
                        output logic dout);

    typedef enum logic [2:0] {S0, S1, S2, S3, S4} state_t;
    state_t state;

    always @(posedge clk or posedge reset) begin
        if(reset) begin
            dout <= 1'b0;
            state <= S0;
        end
        else begin
            case(state)
                S0: begin
                    if(din) begin
                        state <= S1;
                        dout <=1'b0;
                    end
                    else
                        dout <=1'b0;
                end
                S1: begin
                    if(din) begin
                        state <= S2;
                        dout <=1'b0;
                    end
                    else begin
                        state <= S0;
                        dout <=1'b0;
                    end
                end
                S2: begin
                    if(~din) begin
                        state <= S3;

```

```
        dout <=1'b0;
    end
    else begin
        dout <=1'b0;
    end
end
S3: begin
    if(din) begin
        state <= S4;
        dout <=1'b0;
    end
    else begin
        state <= S0;
        dout <=1'b0;
    end
end
S4: begin
    if(din) begin
        state <= S2;
        dout <=1'b1;
    end
    else begin
        state <= S0;
        dout <=1'b0;
    end
end
endcase
end
end
endmodule
```

4 Verification Concepts and Their Implementation

1. Constraint Randomization

- (a) Constraint randomization is used in the transaction class to generate random input bits (din). This ensures that many different input patterns are applied to the DUT, increasing the probability of detecting the sequence 11011, including overlapping cases.

- (b)


```

      rand bit din;
      constraint c { din inside {0,1}; }
      bit detect;
      endclass
      
```

- (c) Design space exploration basically

2. Transaction Class

- (a) A transaction class represents a unit of stimulus sent to the DUT. It stores the input bit and may also include the expected output.

- (b)


```

      class transaction;
      rand bit din;
      bit detect;
      endclass
      
```

- (c) Stimulus becomes an object and the communication between verification components

3. Mailbox Communication

- (a) A mailbox is created in the testbench to transfer transactions between components such as the generator and driver.

- (b)


```

      class transaction;
      rand bit din;
      bit detect;
      endclass
      
```

- (c) gen2drv and mon2scb are two typical mailbox placements in the code, and the .put() and .get() methods help send and retrieve the stimuli and the DUT's output captured via the virtual interface. Mailbox typically helps sync the concurrent transfers between gen2drv and mon2scb.

4. Generator Class

- (a) The generator creates transaction objects, randomizes the input bit, and sends the transaction to the driver through the mailbox.

5. Driver Class with Virtual Interface

- (a) The driver receives transactions from the mailbox and applies the stimulus to the DUT through a virtual interface connected to the DUT signals.

- i. Receive transaction from mailbox
- ii. Wait for clock edge
- iii. Drive din signal to DUT via virtual interface

6. Parallel Execution using fork-join

- (a) The generator and driver operate concurrently using fork-join to simulate a multithreaded testbench environment.

(b)

```
fork
    generator.run()
    driver.run()
join
```

7. Monitor

- (a) The monitor passively observes DUT signals and converts them into transactions. It captures the input bit stream and the detection output and forwards this information to the scoreboard.

8. Scoreboard

- (a) The scoreboard compares the DUT output with the expected result generated by a reference model.
- (b) the scoreboard maintains a 5-bit shift register, checks if the sequence equals 11011, compares expected detection with DUT detection

9. Reference Model

- (a) The reference model represents the correct behavior of the sequence detector. It predicts the expected output based on the input stream.

(b)

```
shift_reg = {shift_reg[3:0], din}

if shift_reg == 11011
    expected_detect = 1
```

10. Immediate Assertions and Concurrent Assertions

- (a) Immediate assertions are used inside procedural blocks to check conditions during simulation. like `assert(detect == 1 when sequence == 11011);`
- (b) Concurrent assertions verify temporal relationships across clock cycles. They ensure that the detection signal is asserted only when the correct sequence appears in the input stream.

11. Functional Coverage

- (a) Functional coverage is implemented to measure how thoroughly the input scenarios have been exercised during simulation.
- (b) Coverage items include:

- i. input bit values
- ii. sequence occurrence
- iii. detection events

12. Covergroups and Coverpoints

- (a) A covergroup is defined to collect coverage information. Coverpoints track the values of signals such as the input bit and detection signal.
 - i. coverpoint din
 - ii. coverpoint detect

13. Cross Coverage

- (a) Cross coverage analyzes combinations of signals, such as the relationship between the input bit and the detection signal.
 - i. cross din, detect

How did we define assertions?

For a non-overlapping 11011 detector, The testbench must prove:

1. Does the sequence 1 1 0 1 1 trigger the output dout = 1?
2. If the input is 1 1 0 1 1 0 1 1, the FSM must reset its search after the first hit. It should not use the last two 1s of the first sequence as the start of the next.
3. For sequences like 1 1 0 1 0 or 1 1 1 0 1 1, Does the FSM correctly recover from a wrong bit without getting permanently stuck?
4. If the reset is asserted in the middle of a sequence (like 1 1 0 → RESET → 1 1), does the FSM successfully flush its memory and start from S0?

1) Why do you need to structure the testbench like this?

Structuring a testbench in this layered, modular fashion is the foundation of modern verification (like UVM - Universal Verification Methodology). We do this for three main reasons:

- It separates the generation of stimulus (Generator) from the actual pin-level toggling (Driver), and separates driving from checking (Scoreboard). This makes the code much easier to debug and maintain.
- By modularizing the components, you can reuse them. For example, if you write an Agent for an I2C interface, you can drop that exact same Agent into a completely different project next year.
- As the Design Under Test (DUT) grows in complexity, a flat testbench becomes a massive, unreadable spaghetti code file. A layered architecture allows multiple engineers to work on different components (e.g., one writes the checker, another writes the sequences) simultaneously.

2) What is the purpose of the Generator, Driver, Monitor, and Scoreboard?

- **Generator:** This component creates high-level "transactions" or "stimulus." Instead of thinking about 1s and 0s on a clock edge, the generator thinks in terms of abstract operations (e.g., "Generate a random 32-bit read packet"). It randomizes these transactions and passes them to the Driver (usually via a mailbox).
- **Driver:** The driver acts as the translator between the abstract testbench and the actual hardware. It receives the high-level transaction from the generator and converts it into pin-level signal wiggles on the interface (e.g., asserting a valid signal, driving the data bus, waiting for a ready signal).
- **Monitor:** This is a passive component. It "listens" to the interface signals without changing them. It watches the pin-level wiggles, groups them back up into a high-level transaction, and broadcasts this transaction to the Scoreboard or coverage collectors.
- **Scoreboard:** This is the "Checker." It receives transactions from the Monitor. It typically contains a reference model (a software version of what the DUT is supposed to do). It compares the actual output of the DUT with the expected output from the reference model and flags an error if they mismatch.

3) Static vs. Dynamic Entities?

- **Static Entities:** These are blocks of code that are allocated in memory at compile/elaboration time and exist throughout the entire duration of the simulation. They cannot be created or destroyed on the fly.
Examples: `module`, `interface`, and the DUT itself.
- **Dynamic Entities:** These are objects whose memory is allocated and deallocated during runtime (simulation). They are created using a constructor (the `new()` function).
Examples: SystemVerilog class objects.

4) Which constituents of the testbench are dynamic and why?

The test, env, agent, generator, driver, monitor, scoreboard, and the transaction items themselves are dynamic.

Why? Because they are implemented using SystemVerilog classes. Using Object-Oriented Programming (OOP) allows us to dynamically create thousands of random transactions, pass them around, and let the garbage collector clean them up. It also allows us to use inheritance (e.g., extending a base test into a specific error-injection test) without rewriting the base code.

5) What is an interface?

An interface in SystemVerilog is a named bundle of signals (nets or variables) used to group related communication lines together.

- **The Problem it Solves:** In traditional Verilog, connecting a complex bus (like AXI or PCIe) between a testbench and a DUT required typing dozens of individual port connections, which is prone to errors and hard to maintain.
- **The Solution:** An interface encapsulates these signals into a single, reusable block. You can pass the entire bundle through a single port connection. It is a static hardware entity, meaning it is created at compile-time and exists physically in the simulation hierarchy.

6) Why do you need a virtual interface in a driver class?

This comes down to the difference between hardware (static) and software (dynamic):

- Modules and interfaces are static. They are wired together when the simulation compiles.
- SystemVerilog Classes (like your Driver) are dynamic. They are created and destroyed during runtime (simulation) using object-oriented programming.
- A dynamic object cannot directly contain or instantiate a static hardware interface. Therefore, the class needs a pointer or handle to point to the physical wires in the testbench. The `virtual` keyword creates this handle. A virtual interface is simply a variable that references a physical interface.

7) What is the purpose of assignment of virtual interface pointer?

The assignment (usually done via the class constructor `new()`) bridges the gap between the software testbench and the hardware DUT. It tells the driver class exactly which physical interface in the testbench it should be manipulating. Without this assignment, the driver is blind and has no physical wires to drive.

8) If you don't use that statement, what will happen?

If you declare a virtual interface handle but forget to assign it to a physical interface, the handle remains null. When your driver tries to execute a statement like `vif.valid = 1;`, the simulation will immediately crash with a "Null Pointer Dereference" (or Fatal Error). It is the equivalent of trying to flip a light switch in a house that hasn't been built yet.

9) What features could you add in a SystemVerilog testbench compared to a Verilog testbench?

SystemVerilog was specifically designed to solve the bottlenecks of traditional Verilog verification. Key features you can add include:

- **Object-Oriented Programming (OOP):** Creating dynamic, reusable classes for transactions, drivers, and monitors instead of static modules.
- **Constrained Random Verification (CRV):** Automating the generation of stimulus using `rand` variables and constraints, rather than hardcoding `#delay` based sequences.
- **Functional Coverage:** Using covergroups to mathematically measure which scenarios have been tested, closing the loop on verification.
- **SystemVerilog Assertions (SVA):** Writing protocol checkers that continuously monitor interfaces for illegal timing or data behaviors.
- **Mailboxes and Semaphores:** Advanced inter-process communication tools to safely pass data between parallel blocks (like a generator and a driver).

Main Comparisons:

SystemVerilog testbenches are Object-Oriented, whereas traditional Verilog testbenches are static and module-based. When building a modern testbench (like verifying an FSM sequence detector), you typically use these OOP concepts:

- **Classes & Objects:** A class acts as a blueprint (e.g., a transaction class defining input bits). Objects are the dynamic instances of that class created during simulation using the `new()` constructor.
- **Inheritance:** You can create a base test class, and then extend it into a `test_error_injection` class, inheriting all the base setup while only modifying specific behaviors.
- **Encapsulation:** Hiding internal variables within a class and only allowing access through specific functions (methods) to prevent unintended modification.

10) Is it possible to define the stimulus class outside the testbench module?

- Yes, it is highly recommended to define your stimulus classes outside the `module ... endmodule` block. They are typically placed in separate `.sv` files or wrapped inside a package.
- **Can you still randomize and use it inside the testbench?** Absolutely. Once defined outside, you can import the package or `#include` the file containing the class. You then declare a handle, instantiate the object using `new()`, and call `.randomize()` on it directly within your top-level testbench module's initial block.
- **Which approach enhances code reusability?** Defining the stimulus class outside the testbench module drastically enhances reusability. If you define a class inside a module, its scope is locked to that specific module. By defining it externally in a package, you can reuse the exact same transaction class across multiple different projects—for example, using the same transaction definition for block-level simulation and later for full System-on-Chip (SoC) verification.

11) What are the different ways to randomize stimulus generation (\$urandom vs object.randomize())?

- **\$urandom:** This is a system function that returns a 32-bit unsigned pseudo-random integer. It is procedural, does not understand classes, and is best used for simple, unconstrained random numbers (like generating a random delay counter).
- **object.randomize():** This is a built-in OOP method called on a class instance. When called, the constraint solver evaluates all the **rand** or **randc** variables within that specific object simultaneously and solves any defined constraint blocks to generate valid, legal scenarios.

12) What is constraint randomization?

Constraint randomization allows you to restrict the random values generated to a valid or targeted subset. Instead of testing purely random values (which might generate illegal opcodes or useless data), you set rules. For example, you can constrain an input bit stream to favor certain sequences to hit transition edge cases faster.

Example:

```
class packet;
  // Defining random variables
  rand logic [7:0] data;
  rand logic      valid;

  // Defining a constraint block
  constraint valid_data_range {
    data > 8'h0A;
    data < 8'hF0;
    valid == 1'b1;
  }
endclass
```

13) Coverage and Verification Closure: Will randomization always guarantee 100% functional coverage?

No. Pure randomization forms a "bell curve" of coverage. It will hit the bulk of standard operations very quickly, but rare corner cases, deep FIFO full conditions, or complex error recovery states require exponential time to hit by pure chance.

14) Is Direct test + constraint randomization a way to attain 100% functional coverage?

Yes, this is the standard industry methodology. You start with Constrained Random Verification (CRV) to achieve 80-90% coverage rapidly. Then, you analyze the coverage report to identify the "holes" (uncovered bins). Finally, you write highly constrained random tests (directed-random) or pure directed test vectors to target those exact corner cases until you hit the 100% target.

15) Why do you need coverpoints and cross coverpoints?

- **Coverpoint:** Tracks the values a single variable takes during simulation (e.g., verifying an FSM reached all of its defined states).
- **Cross Coverpoint:** Tracks the combination of two or more coverpoints occurring simultaneously. This is critical because hitting all possible states, and hitting all possible input commands separately, does not guarantee you tested every command while inside every state. Cross coverage ensures complex, concurrent interactions are fully verified.

16) What are assertions?

Assertions (specifically SystemVerilog Assertions or SVA) are blocks of code that continuously monitor signals over time to ensure that specific rules (protocols) are never broken.

Usually found in Interface, Why? Because assertions need to look at the raw, cycle-by-cycle hardware signals (the physical pins wiggling). The interface is the static hardware bundle that connects to the DUT, making it the perfect place to embed these continuous timing checks. You generally do not put them in dynamic software classes like the Generator or Driver.

17) How many mailboxes are used in the architecture? What is the purpose of each one?

Based on standard architecture, two mailboxes are typically used:

- **Mailbox 1 (Generator to Driver):** Its purpose is to take the randomized, high-level transactions created by the Generator and pass them to the Driver so they can be converted into pin-level stimulus.
- **Mailbox 2 (Monitor to Scoreboard):** Its purpose is to take the transactions that the Monitor has observed on the bus and pass them to the Scoreboard for verification against expected results.

18) How did you execute concurrent tasks in testbench? Which are those concurrent tasks?

Concurrent tasks are executed using the `fork ... join` (or `fork ... join_any` / `fork ... join_none`) construct in SystemVerilog. The main concurrent tasks are typically the `driver.run()` and `monitor.run()` tasks. These must run simultaneously in parallel because the driver needs to continuously send data while the monitor needs to continuously watch the bus.

19) Mailbox handle is passed from generator to driver... Copies of mailbox is not created. Justify the statement.

In SystemVerilog, objects are passed by reference (using handles/pointers). You instantiate a single physical mailbox in the env or agent memory space using `new()`. You then pass that exact same handle to both the generator and driver. If you created a copy, the Generator would put transactions into its own private box, and the Driver would be waiting forever looking inside a completely different, empty box. They must share the exact same memory location to communicate.

20) Which components in test bench architecture used virtual interface and why?

The Driver and the Monitor use virtual interfaces. These components are dynamic software classes created during simulation. They cannot directly connect to static hardware wires. A virtual interface acts as a software pointer that bridges the gap, allowing the dynamic classes to see and manipulate the static hardware pins defined in the physical interface.

21) How will you differentiate transaction and interface?

- **Interface:** A static, physical bundle of hardware wires (like `clk`, `reset`, `data_in`, `write_en`) that connects modules together.
- **Transaction:** A dynamic software object (a class) that represents a high-level abstract operation (like a "Read Packet" or a "Write Packet"). It contains variables, not physical wires.

22) How functional verification is done in scoreboard?

The scoreboard contains a "reference model" (often a simple SystemVerilog queue acting as a perfect, software version of a FIFO). When a write transaction occurs, the scoreboard pushes that data into its software queue. When a read transaction is observed by the monitor, the scoreboard pops the expected data from its software queue and compares it against the actual data the DUT provided.

23) If the FIFO outputs the wrong order, which part of the testbench will catch it?

The Scoreboard. Because the scoreboard's reference model is keeping track of the exact order data was written in, if the monitor sends read data that is out of sequence, the scoreboard's comparison logic will flag a mismatch error.

24) "Functional coverage tracks whether all intended scenarios of the DUT are exercised. Assertions check correctness. Functional coverage measures verification completeness." Justify the statement.

This statement perfectly captures the dual nature of verification.

- **Assertions** are strictly about correctness. They ask, "Did the DUT do anything illegal?" If you run a simulation for a million clock cycles and no assertions fire, you know the DUT didn't break any rules.
- However, what if your testbench only ever tested the FIFO while it was empty? You didn't break any rules, but you didn't test much either. This is where **Functional Coverage** comes in. It asks, "Did we actually test every feature?" (Completeness). It tracks if you hit the full state, the empty state, concurrent read/writes, etc.

You need both: Coverage to ensure you tested everything, and Assertions/Scoreboards to ensure everything you tested worked correctly.

25) Always ask yourself what are you functionally verifying in order to do the best SVA!

26) The Types of Assertions in SystemVerilog?

SystemVerilog has two distinct types of assertions: Immediate and Concurrent.

- **Immediate Assertions:** These are simple, single-instant checks. They act exactly like an `if` statement in software. You place them inside procedural blocks (like `always`, `initial`, tasks, or functions). They do not understand time or clock cycles; they just check if a condition is true right now.

Syntax: `assert (a == b) else $error("Mismatch!");`

- **Concurrent Assertions:** These are complex, time-aware checks. They are tied to a clock edge and continuously monitor the relationship between signals over multiple cycles. They run in the background as separate hardware threads.

Syntax: `assert property (@(posedge clk) a |=> b);`

27) What kind of assertion did we use for the FSM?

In the `interface.sv` file we just built, we exclusively used Concurrent Assertions. Here is why:

- **The `@(posedge clk)` definition:** We explicitly told the assertion to evaluate only on the rising edge of the clock.
- **The property block:** We defined the rules using `property ... endproperty` blocks, which are specifically designed for concurrent temporal logic.
- **The Implication Operator (`|=>`):** This operator is a hallmark of concurrent assertions. It evaluates a precondition (e.g., "is reset high?") and, if true, checks the consequent (e.g., "is dout 0?").

28) Demo the fork-join

```
task run();
  fork
    gen.run(); // Starts generating 100 random bits
    drv.run(); // Starts driving bits to the FSM
    mon.run(); // Starts watching the FSM's outputs
    sb.run();  // Starts scoring the results
  join_none
endtask
```

Why is it in `env`?

SystemVerilog executes code sequentially by default. If we didn't use `fork`, the simulator would start the Generator, and the Driver, Monitor, and Scoreboard would sit completely frozen waiting for the Generator to finish all 100 transactions before they even woke up.

`fork` splits the execution into four parallel threads. All four components wake up and start running their `run()` tasks simultaneously, allowing them to pass data back and forth through the mailboxes in real-time. We used `join_none` so that the `env.run()` task could spawn these threads in the background and immediately return control to the `testbench_top` module.

29) Where do you put covergroups?

The standard industry practice is to put the covergroup inside the **Monitor** (`monitor.sv`). **Why?** The Monitor is already connected to the virtual interface and its entire job

is to passively watch the pins every clock cycle. It doesn't generate data, and it doesn't judge if the data is correct. It just records what happens. This makes it the perfect, non-intrusive place to collect statistics!

30) Coverage vs. Cross Coverage

- **Coverage (Coverpoints):** Tracks individual items. Example: Did `din` ever equal 1? Did we ever see the sequence `1 => 1 => 0 => 1 => 1`?
- **Cross Coverage:** Tracks the mathematical intersection of two or more coverpoints. It asks: "Did event A happen at the exact same time as event B?"

The perfect Cross Coverage for our FSM: Because you designed a Pure Mealy machine for the sequence 11011, the output (`dout`) can only go high if the current input (`din`) is a 1. It is physically impossible for `din` to be 0 and `dout` to be 1 at the same time. We can use Cross Coverage to track all combinations of `din` and `dout`, and explicitly tell the simulator to ignore that impossible combination!

Improvements :

1. How do we extend this test bench to a UVM test bench? What will all things change and what will stay intact? Will transitioning to UVM help? (does it always help? if not then when?)
2. Which object oriented programming aspects were covered and where (with example)?
3. How would the verification differ for overlapping case (with overlapping allowed for 1 bit/2 bits/ more)?
4. How exactly is "rand" helping us improve search space?
5. Sometimes there are errors with assertions when done concurrently, when do they occur and how does one solve them?
6. Where are sequence detectors useful? (why is their functional verification important?)

Answers :

Concurrent assertions evaluate continuously over clock cycles. When they fail, it's usually a timing mismatch between your brain and the simulator.

Floating "X" States at Startup: When the simulation starts at Time = 0, nothing is powered on. Signals are floating. If an assertion checks the pins before the reset finishes, it crashes.

Solution: Always use `disable iff (reset)` to tell the assertion to stay asleep until the system is initialized.

The "Off-by-One" Cycle Error: People frequently confuse Overlapping Implication (`|->`) with Non-Overlapping Implication (`|=>`).

Solution: If the output is supposed to react on the exact same clock edge (like our Mealy machine), use `|->`. If the output takes a full clock cycle to process through a flip-flop (like a Moore machine), you must use `|=>` so the assertion waits a cycle before checking.

Sampling Dirty Logic (Delta Cycles): If the RTL uses blocking assignments (`=`) instead of non-blocking (`<=`) for sequential logic, the signals will glitch right on the clock edge.

Solution: Ensure the RTL design strictly adheres to proper Verilog coding standards so the assertions sample clean, stable data in the "Preponed" region of the clock cycle.

For Overlapping detections :

The Scoreboard Change: In your current non-overlapping scoreboard, you flush the buffer: `if (found) shift_reg = 5'b00000;`. For an overlapping detector, you remove the flush. The shift register keeps all its old data. If the sequence 11011011 comes in, the scoreboard will expect `dout` to spike high twice (once on the 5th bit, once on the 8th bit).

Coverage Bins: You must explicitly create a sequence bin for $(1 \Rightarrow 1 \Rightarrow 0 \Rightarrow 1 \Rightarrow 1 \Rightarrow 0 \Rightarrow 1 \Rightarrow 1)$ to prove the overlapping recovery actually happened in simulation.

State Machine Transitions: You must verify that hitting the final 1 transitions the machine to S2 (since the last two bits 11 act as the start of a new sequence) rather than kicking it all the way back to S0.